STS EEG Screening Server Update

Global STD fix + image cascades — September 2026

Four steps on each server. Do the development server first; when its results look right, do production the same way. Everything is a file copy through your normal Remote Desktop session — no scripts required.

Each study is now processed in **two passes**: the brain panel is written first, within minutes, and the image cascade follows. The panel reaches the practitioner without waiting for the cascade, and a cascade failure can no longer destroy it.

What this changes

- **Global STD** on page 1 of every Brain Panel currently prints `1000.00` with `z 2642.71`. After the update it reads about `2.5`, inside the normal range. No other metric moves — verified 47 of 48 identical against this server's own output.
- **An image cascade** (`<study>.imagecascade.pdf`, two pages per ICA component: the component view and its brain source-localization view) is written next to every panel, a few minutes after the panel itself.
- **Brain pages without OpenGL.** They are now drawn in software, so they work on the AWS servers. The old 3D viewer needed a graphics card; `CLEANEEG_NO_BRAIN` is no longer needed.
- **Disk:** roughly 21 MB per study with the brain pages. Production has ~200 GB on a separate data disk: 1,000 studies use about 21 GB of it. Cascades are written next to the uploaded EDF, so this assumes the upload folder is on that disk — the watchdog's `GB free` figure shows which drive is in use. It warns below 10 GB.

The files

On your workstation, open this folder in Explorer:

```
C:\BrainPanel\clean_eeg_project 2025\deploy\server-2026-09\staged
```

File	Goes into	What it does
`files\edftotextbynameplotproc.py`	project\files\	Global STD fix
`files\edftotextbycommandplotproc.py`	project\files\	panel pass only
`files\Montage_6.py`	project\files\	cascade rendering
`files\dummy_gui.py`	project\files\	cascade component pages
`files\Component_selector.py`	project\files\	same fixes, kept in sync
`files\color_strip.py`	project\files\	new — faster component pages; the two above need it
`files\create_report_pdf.py`	project\files\	cascade screen capture
`files\deidentify.py`	project\files\	new — masks names on cascade pages
`process\brain_render.py`	project\process\	new — brain pages, no OpenGL

process\offscreen_tk.py	project\process\	new — component pages without the screen
mne_data\ (whole folder)	project folder (next to module7.py)	brain surfaces, ~25 MB; merge
run_cascade.py	project folder (next to module7.py)	new — the cascade pass
tomwatchdog_serialized.py	project folder (next to module7.py)	new watchdog — one at a time
staged-optional\process\detect_artifact.py	project\process\	quieter panel log; metrics unchanged

"Project folder" is the directory containing `module7.py`. On the development server that is `C:\app\MyCleanEEG\CleanEEGProject`.

On the server

In your Remote Desktop session to the server.

1. Back up the seven files being replaced

In the project's `files\` folder, make a new folder called `backup-2026-09` and copy these six files into it:

```
Component_selector.py
Montage_6.py
create_report_pdf.py
dummy_gui.py
edftotextbycommandplotproc.py
edftotextbynameplotproc.py
```

Also copy `process\detect_artifact.py` into the same backup folder.

2. Paste the new files in

- Copy the eight `.py` files from `staged\files\` into the project's `files\` folder. Say **Yes** to overwrite. Two are new: `deidentify.py` and `color_strip.py` — without `color_strip.py` the cascade cannot start.
- Copy both files from `staged\process\` (`brain_render.py`, `offscreen_tk.py`) into the project's `process\` folder.
- Copy `staged-optional\process\detect_artifact.py` into the project's `process\` folder too, and say **Yes** to overwrite. It silences the alpha debug prints that flood the panel log; all 48 metrics were verified identical.
- Copy the `staged\mne_data` folder into the project folder. If the project already has one, let it merge and say **Yes** to overwrite — the six surface files are identical to the ones already there.
- Copy `run_cascade.py` and `tomwatchdog_serialized.py` into the project folder itself, next to `module7.py`.

3. Swap the watchdog

In the console window where `tomwatchdog.py` is running, press **Ctrl-C**. Then, in that same window:

```
set CLEANEEG_NO_BRAIN=0
set CLEANEEG_OFFSCREEN=1
```

The first line makes sure the brain pages are **on**. An earlier version of these steps set `CLEANEEG_NO_BRAIN=1` on the servers because the old brain pages needed OpenGL. They no longer do, so the variable must be 0 or unset.

The second line is required. It draws the component pages in memory. Without it the cascade photographs the desktop, so a disconnected Remote Desktop session, a lock screen or a sleeping display gives black or wrong pages with no error. Both lines cover this console window only — start the watchdog from the same window.

Now start the new watchdog. **Development server:**

```
py tomwatchdog_serialized.py "c:/app/STSEEGScreening/Source/Practitioners"
```

Production server:

```
py tomwatchdog_serialized.py
```

If the window answers `'py' is not recognized` (production has no py launcher), run the same Python by its full path instead — on production:

```
c:\users\tcollura\appdata\local\programs\python\python312\python
tomwatchdog_serialized.py
```

It prints `monitor dir:` and `interpreter:` first — glance at both. The production path is built in; the development server needs its own path given.

It must run in that console window — not as a service, not from a scheduled task. The cascade captures the screen, and only that session has one.

If you ran `setx CLEANEEG_NO_BRAIN 1 /M` on this server before, undo it from an elevated prompt: `setx CLEANEEG_NO_BRAIN 0 /M`.

4. Upload one study and check the result

The panel comes first; the cascade follows. Panels and cascades run side by side, so a new panel never waits behind an earlier cascade. The console (and `logs\watchdog-YYYY-MM-DD.log`) shows something like:

```
14:02:11  QUEUED (created): ...\study.edf
14:02:17  PANEL START: ...\study.edf  (panels waiting 0, ...)
    P| [offscreen] ON - component page composited with Agg, ...
14:06:39    PANEL in 262s  rc=0  ...
14:06:39  CASCADE START: ...\study.edf  (cascades waiting 0)
    C| [offscreen] ON - component page composited with Agg, ...
    C| CASCADE OK: ...\study.imagecascade.pdf (30961075 bytes)
15:29:43    CASCADE in 4984s  rc=0  ...
```

`P|` lines come from the panel pass, `C|` from the cascade pass. **Check for** `[offscreen] ON`**.** If it says `OFF`, Ctrl-C, run `set CLEANEEG_OFFSCREEN=1` in that window and start the watchdog again.

Copy log text from `logs\watchdog-*.log`**, not from the console.** Selecting text in a Windows console pauses all output to it, and that pause freezes the running panel or cascade until the selection is cleared — no error, no new files. The watchdog now switches this off for its own window (it prints `console QuickEdit: OFF`) and writes to the console from a separate thread, so even a Ctrl+A selection only pauses the window: the study keeps running and the log keeps every line. Clear a selection with **Esc** (title starts with **Select**). Never press Ctrl+C there with nothing selected — it stops the watchdog and the running study.

Those timings are from the development server before the October 2026 speed-up: panel 2–5 minutes, cascade about 80 minutes for a 10-minute recording. Each cascade component is now roughly 4× faster. Then check the files next to the EDF:

Dev	Prod	Check
☐	☐	`.icale.rep.pdf` — page 1, the **Global STD** row reads about **2.5**, not 1000.
☐	☐	`.imagecascade.pdf` exists, appearing a few minutes after the panel.
☐	☐	Open the cascade and page through it: component graphs on every page, **no black pages**.
☐	☐	After each component page there is a **brain page**: shaded cortex from six sides with the source in red–yellow, plus three slices.
☐	☐	Page 1 again: the four **Moment 3** rows are in the hundreds to low thousands.

If the Moment 3 rows are in the **tens of thousands**: that server has numpy 2.x and `detect_artifact.py` from step 2 did not land in `process\`. Copy it and re-run the study.

That is the whole deployment. Production is the same four steps, starting the watchdog with no argument, in the console session kept logged in from the development server.

If something is wrong

1. Copy the files from `backup-2026-09` back: `detect_artifact.py` into `process\`, the other six into `files\`. The new `brain_render.py` and `mne_data` can stay; nothing else uses them.
2. Ctrl-C the new watchdog.
3. Start the old one: `py tomwatchdog.py`

To keep the panel fix but stop cascades, set `CASCADE_PASS = False` near the top of `tomwatchdog_serialized.py` and restart it. Panels are unaffected.

Nothing in this update touches data files, the reference database, or any existing output, so rollback is a complete undo.

Worth knowing

- **Why two passes:** the cascade renders inside the same code that builds the panel, so in one pass the panel came last — and a cascade crash destroyed it, which is exactly what the OpenGL failure did on the dev server. Split in two, the panel is on disk before the cascade starts.
- **Why the watchdog changes:** the old one starts a new process for every upload with no limit. The new one waits for an upload to finish writing, ignores duplicate events for the same file, runs panels alongside cascades but cascades one at a time, and kills a panel past 10 minutes or a cascade past 4 hours.
- **Screen:** with `CLEANEEG_OFFSCREEN=1` the cascade never reads the screen, so disconnecting from the server, a lock screen or a screen saver does not affect it. Without the variable all three produce black or wrong pages.
- **Older panels:** every panel issued before this update carries the bogus Global STD row. Nothing else on them is wrong; their out-of-bounds count is one too high.
- **Longer documents:** README.md explains each change. MESA_OPENGL.md is out of date: the brain pages no longer need OpenGL.